

Article

Semantic-Driven Context Pruning for Arabic RAG Systems: Toward Memory-Efficient vLLM-Based Deployment

Ali Abdul Razzaq Tarish ^{1,*}¹ College of Computer Engineering, University of Technology - Iraq, Baghdad, Iraq

* Correspondence: akramtaha30@gmail.com

Abstract

Retrieval-Augmented Generation (RAG) for Arabic is constrained by a memory bottleneck: Arabic's rich morphology inflates token counts relative to English, enlarging the Key-Value (KV) cache that memory-efficient serving engines such as vLLM must hold per request. We introduce a Lightweight Semantic Pruning Middleware (LSPM) that scores retrieved passages at the sentence level with a cross-encoder and retains only the most query-relevant subset at a fixed or dynamic compression ratio. Across 30 verified Arabic/English sentence pairs, two tokenizers confirm a mean per-pair token ratio of 1.79x and 1.72x (bootstrap $p < 0.0001$ vs. parity). On 140 real Arabic Reading Comprehension Dataset (ARCD) questions (980 live generations), LSPM significantly outperforms naive sentence-count-matched truncation on token-F1 at the most aggressive compression ratio tested ($r = 0.3$: $+0.114$ F1, Holm-Bonferroni-adjusted $p = 0.0054$) while remaining statistically equivalent to the unpruned answer (TOST, $p = 0.038$); naive truncation is significantly worse than the unpruned answer (adjusted $p = 0.0004$). A TF-IDF end-to-end retrieval check and a blind two-rater human evaluation directionally reinforce this result. A small, single-run, acknowledged-confounded GPU diagnostic is reported separately as a preliminary systems observation, not a validated deployment claim; missing token-budget-matched and modern compression baselines are disclosed as open limitations. We release the implementation, evaluation code, and datasets described in this paper.

Keywords: retrieval-augmented generation; Arabic natural language processing; prompt compression; vLLM; large language models

Academic Editor: Firstname
Lastname

Received: date
Revised: date
Accepted: date
Published: date

Copyright: © 2026 by the authors.
Submitted for possible open access
publication under the terms and
conditions of the [Creative](#)

1. Introduction

Large Language Models (LLMs) [1] have become the default reasoning engine behind Retrieval-Augmented Generation (RAG) systems that ground generation in externally retrieved evidence rather than relying solely on parametric memory [2]. Every retrieved token that enters an LLM's context window occupies a slot in the Key-Value (KV) cache that memory-efficient serving engines such as vLLM must hold in GPU memory for the duration of the request [3]. vLLM's PagedAttention algorithm manages this pressure by paging the KV cache into fixed-size blocks [3], and related techniques such as attention-sink streaming [4] and heavy-hitter eviction [5] further manage cache consequences after tokens have already entered it. None of these techniques reduce the number of tokens that create the cache in the first place; that is the gap this paper addresses for Arabic specifically.

Arabic is a morphologically rich, templatic language, and standard subword tokenizers, trained predominantly on English-dominated corpora, fragment Arabic text substantially more than English text of equivalent meaning. For a fixed context budget this means an Arabic RAG system retrieves less semantic content than an English one, and for a fixed amount of retrieved content it imposes a larger KV-cache footprint. Prompt-compression methods such as LLMingua [6], Selective Context [7], and RECOMP [8] address context size generally but are evaluated largely on English text, without Arabic tokenization disparity or vLLM's KV-cache mechanics as first-class design constraints. This context-size problem is compounded by evidence that LLMs do not use long contexts uniformly: performance degrades when relevant information is buried in the middle of a long context rather than at its start or end [9], which motivates pruning irrelevant content rather than simply retrieving more of it.

We propose a Lightweight Semantic Pruning Middleware (LSPM), a thin layer between retriever and generator that (i) splits retrieved passages into sentences, (ii) scores each sentence's relevance to the query with a cross-encoder [10], and (iii) reconstructs a compact, order-preserving context from the highest-scoring sentences, at a ratio that can be fixed or computed dynamically from vLLM's live /metrics endpoint.

This paper is principally an architecture and preliminary-validation study of context selection for Arabic RAG: the tokenization measurement, the naive-truncation baseline comparison, the end-to-end retrieval check, and the human evaluation are its central evidence. It additionally includes one narrowly scoped GPU benchmark, reported separately in Results as a preliminary systems diagnostic rather than a validated deployment result: only its single lowest-concurrency cell ($c = 1$) shows a KV-cache-occupancy difference, and even there LSPM shows an 8.5-11.9% lower request throughput alongside a near-unchanged completion-token rate; the comparison is already mixed at $c = 10$ and reverses by $c \geq 25$, a pattern we attribute to a specific, disclosed methodological confound (load-generator-side scoring cost; no server restart between method blocks). The scope and limits of every claim in this paper, measured, analytical, and not-yet-validated alike, are stated explicitly in the sections that follow.

Contributions: (1) A concrete system architecture and open reference implementation for sentence-level, cross-encoder-driven context pruning for Arabic RAG on vLLM-class serving infrastructure, including a dynamic compression-ratio controller design coupling pruning aggressiveness to real-time GPU KV-cache occupancy (implementation feature, not itself independently validated against live traffic). (2) A quantitative measurement of Arabic-English tokenization disparity on a 30-pair hand-verified parallel corpus (bootstrap $p < 0.0001$ for both of two tokenizers tested). (3) A real, paired baseline comparison against naive sentence-count-matched truncation, re-tested at increasing scale up to a 140-question

ARCD re-test (980 live generations) sized from a post-hoc power calculation on an earlier 40-question pilot's effect size, resolving the head-to-head comparison in LSPM's favor at the most aggressive ratio ($r = 0.3$) after Holm-Bonferroni correction, while leaving it open at more moderate ratios. (4) A complementary end-to-end retrieval check that removes the answer-guaranteed pool construction, with a real TF-IDF retriever ($\text{recall@6} = 86.7\%$), reproducing the same directional advantage. (5) An analytical, explicitly-labeled KV-cache savings projection and a preliminary, single-run GPU throughput/TTFT/KV-cache diagnostic against a self-hosted vLLM server that tests it, reported as a single-cell exploratory observation with an identified and disclosed methodological confound at higher concurrency, not as a validated systems result. (6) A blind, two-rater human evaluation (90 answers, correctness and faithfulness, substantial-to-near-perfect inter-rater agreement) that directionally corroborates the automatic-metric result.

The remainder of this paper is organized as follows. Section 2 reviews related work in context compression, retrieval, and Arabic natural language processing, and positions LSPM's contribution relative to it. Section 3 describes LSPM's architecture, the experimental setup, and the reproducibility details needed to replicate every reported result. Section 4 reports the quantitative results: tokenization disparity, the ARCD baseline comparison, the end-to-end retrieval check, the human evaluation, and the preliminary GPU diagnostic. Section 5 discusses what these results do and do not establish, states the study's limitations explicitly, and outlines the experiments required before a systems-level deployment claim would be justified. Section 6 concludes.

2. Related Work

RAG was introduced by Lewis et al. as a way to combine a parametric generator with retrievable, non-parametric memory for knowledge-intensive NLP tasks [2]; our system is agnostic to the RAG variant used upstream and operates purely on retrieved passages. Dense retrieval methods such as DPR [11] and ColBERT [12] are complementary to LSPM's retriever-agnostic design. The closest line of work is prompt and context compression: LLMingua uses an auxiliary LM to iteratively remove low-information tokens under a budget controller [6]; Selective Context uses self-information to prune redundant content [7]; RECOMP compresses retrieved documents into extractive or abstractive summaries [8]. None of these three works reports a comparison against a naive sentence-count-matched truncation baseline, a gap our baseline comparison begins to address for the sentence-scoring family specifically, and none targets Arabic tokenization disparity or vLLM's KV-cache mechanics explicitly. LSPM differs by performing sentence-level relevance filtering via a cross-encoder [10], [13] scored directly against the query rather than token-level self-information or learned summarization, and by closing a feedback loop with vLLM's live KV-cache occupancy metric that has no analogue in this prior work.

Cross-encoder rerankers have been a staple of the modern retrieval pipeline since Nogueira and Cho showed a BERT-based reranker could substantially improve passage ranking [10]; Sentence-BERT reformulated this family of models for efficient semantic similarity computation [13]. Multilingual models such as BGE-M3 [14] extend this capability to over 100 languages including Arabic, and LSPM's reference implementation supports a BGE-reranker backend as a pluggable, higher-accuracy option; the specific cross-encoder used to produce every result reported in this paper is cross-encoder/mmarco-mMiniLMv2-L12-H384-v1 (Section 3). Arabic NLP has matured substantially with the arrival of large pretrained models specific to the language: AraBERT adapted the BERT pretraining recipe to a large Arabic corpus [15], and the Arabic Reading Comprehension Dataset (ARCD) [16], a human-authored, SQuAD-style benchmark of roughly 1,400 question-answer pairs over Arabic Wikipedia articles, remains a standard benchmark for Arabic machine reading comprehension and is the dataset our ground-

truth re-test uses. None of this substantial body of Arabic NLP work, to our knowledge, has been connected to the systems-level question of KV-cache-efficient serving, which is the specific contribution of this paper.

On the systems side, vLLM's PagedAttention [3] builds on continuous-batching, iteration-level scheduling [17], and cache-management techniques such as StreamingLLM's attention sinks [4] and H2O's heavy-hitter eviction [5] operate orthogonally to LSPM: they manage tokens already in the KV cache, whereas LSPM reduces the number of tokens handed to the engine in the first place, so the two families compose naturally. Our fidelity evaluation relies on three complementary automatic metrics: BLEU measures n-gram precision overlap, ROUGE-L uses longest-common-subsequence overlap, and BERTScore computes similarity using contextual embeddings rather than exact token overlap, correlating more closely with human judgments of semantic equivalence. For the confidence intervals and paired significance tests used throughout this paper, we use the nonparametric bootstrap [18], which avoids distributional assumptions that would be difficult to justify at our sample sizes.

Positioning this work against recent LLM-oriented systems papers is instructive. Setiawan and Soewito [19] build a two-agent RAG pipeline (a Threat Analysis Agent and a Snort Rule Agent) that retrieves Common Weakness Enumeration entries and generates intrusion-detection rules, and report that instruction-based chain-of-thought prompting raises RAGAS faithfulness from 0.556 to 0.769 and Snort-rule precision to 1.00; their evaluation targets generation quality and rule correctness under a fixed retrieval pipeline, whereas this paper targets the retrieved-context size itself and its downstream KV-cache cost, a complementary rather than competing axis. Vaidya et al. [20] propose an evolutionary phylogeny (H-Tree) for hallucination-risk scoring, treating paraphrase drift as cumulative mutation and reaching 96% accuracy on TruthfulQA; their concern is output reliability after generation, while LSPM intervenes earlier, at context construction, and the two approaches could in principle compose (pruned context feeding a hallucination-risk-scored generation stage). Kataria [21] reports a hierarchical multi-agent LLM framework for site-reliability incident analysis, coordinating five domain-specialized agents via the Model Context Protocol and achieving 92.1% root-cause accuracy against a 67.3% manual baseline; like this paper, it is a systems-oriented LLM application with a measured infrastructure benchmark, but its evaluation runs on production-scale chaos-engineering testbeds with hundreds of injected failures, a considerably larger validation effort than the single-GPU, single-run benchmark reported here (Section 4), a scale gap this paper's Limitations and Future Work discussion (Section 5) addresses directly rather than obscures.

Positioning statement. Relative to this prior work, LSPM's specific claim to novelty is narrow and stated precisely: it is the first system, to our knowledge, that (i) performs sentence-level, cross-encoder-driven context pruning specifically evaluated on Arabic RAG, where tokenization inflation makes the KV-cache cost of retrieved context measurably worse than for English, and (ii) couples pruning aggressiveness to a live serving-engine signal (vLLM's KV-cache occupancy) rather than a static, offline-computed budget. It does not claim to be a stronger compressor than LLMingua, Selective Context, or RECOMP in an absolute sense; that comparison is not yet made in this paper (Section 5, Limitations) and is the most important open validation step. What this paper does establish is that, within its own paired comparison, sentence-level semantic selection is a better default than naive truncation for Arabic RAG at aggressive compression, and it reports that result, and only that result, without overstating it into a general compression or deployment claim.

3. Materials and Methods

3.1. System Design

Figure 1 shows LSPM's pipeline. A user query is embedded and matched against a vector database (Chroma) to retrieve a candidate passage pool. LSPM splits each retrieved passage into sentences, pairs every extracted sentence with the query, and scores it with a cross-encoder; the reference implementation supports a fast MiniLM-family backend for interactive use or a BGE-reranker-v2-m3 backend for maximal accuracy as a configurable choice, and cross-encoder/mmarco-mMiniLMv2-L12-H384-v1 specifically is the model used to produce every result reported in this paper (Section 3.3). This scoring step has a measured mean latency of 506.1 ms. The highest-scoring sentences are retained, in their original order, up to a target compression ratio r (the fraction of sentences kept), and reassembled into a compact context that is handed to the generator running behind a vLLM server. An optional dynamic controller can adjust r at request time from vLLM's live /metrics KV-cache occupancy: as occupancy rises toward a configured threshold, r is tightened; as it falls, r is relaxed. This controller is an implementation feature of the released system and is described here as a design contribution; it is not independently validated against live, time-varying traffic in this paper, and that validation (fixed vs. dynamic r under sudden concurrency changes, measuring controller stability, oscillation, achieved token budgets, and resulting answer quality) is identified explicitly as required future work (Section 5).

3.2. Why Semantic Pruning Should Preserve Answer Quality Better Than Truncation

The mechanistic rationale for expecting LSPM to outperform positional truncation, particularly at aggressive compression, follows directly from where answer-bearing content tends to sit in a retrieved passage pool relative to where a fixed-position cut falls. Naive truncation keeps the first $r \cdot N$ sentences of the concatenated retrieved pool regardless of their content; if the answer-bearing sentence for a given question falls after that cut point, which becomes increasingly likely as r shrinks and the retrieval pool mixes gold and distractor passages, truncation removes it outright and no downstream generation strategy can recover it. LSPM instead ranks every sentence by a cross-encoder's estimate of its relevance to the specific query, so the retention decision is conditioned on content rather than position: an answer-bearing sentence that would fall outside a positional cut can still be retained if it scores above the compression threshold. This predicts a specific, testable pattern, namely that LSPM's advantage over truncation should widen as r shrinks (more content is discarded, so position becomes an increasingly poor proxy for relevance) and should be concentrated on questions where the gold sentence is not near the start of the retrieved pool. The result reported in Section 4 is consistent with the first half of this prediction: the significant head-to-head advantage for LSPM is concentrated at the most aggressive ratio tested ($r = 0.3$) and is not significant after correction at $r = 0.5$ or $r = 0.7$, exactly where truncation's positional cut is least likely to exclude the answer in the first place. We did not directly test the second half of this prediction, position of the gold sentence within the retrieved pool, as an explicit moderating variable in this study; doing so, and reporting whether LSPM's advantage is in fact concentrated on late-positioned gold sentences, is identified as a direct, low-cost follow-up analysis on the existing ARCD re-test data (Section 5).

3.3. Experimental Setup

Tokenization disparity: a hand-verified, 30-pair Arabic-English parallel corpus was tokenized with two contemporary tokenizers (Qwen2.5-7B-Instruct [22], Llama-3.1-8B-Instruct [23]), and a one-sample bootstrap test (10,000 resamples) tested the mean ratio against parity. Two related quantities are reported for this disparity in this paper and should not be conflated: the mean per-pair token ratio (each pair's Arabic token count

divided by its English token count, then averaged across the 30 pairs, reported in Section 4.1 and Table 1 as 1.79x for Qwen2.5 and 1.72x for Llama-3.1) and an aggregate ratio (total Arabic tokens across all 30 pairs divided by total English tokens across all 30 pairs, reported in the Abstract and Conclusions as approximately 1.76x-1.69x across the two tokenizers). The two are related but numerically distinct aggregation procedures; both are computed on the same underlying 30-pair corpus and both reject parity at $p < 0.0001$, but a per-pair mean and a ratio-of-sums are not required to be numerically identical, and this paper reports both explicitly rather than treating them as interchangeable.

Fidelity pilot ($n = 8$): a small pilot compared LSPM against raw (unpruned) context and naive truncation at r in $\{0.3, 0.5, 0.7\}$ using BLEU, ROUGE-L, and BERTScore-F1 against the unpruned answer as a reference.

3.4. ARCD Ground-Truth Re-Test

ARCD ground-truth re-test ($n = 140$): to address the small pilot's scale and its scoring against the unpruned answer rather than a real gold answer, we sampled 140 questions from ARCD's validation split [16] (seed = 42), built a six-document pool per question guaranteeing the gold paragraph is present (recall@6 = 100% by design) plus five distractors from unrelated articles, and generated seven answers per question against the live Llama-3.1-8B-Instruct endpoint: raw (unpruned), LSPM at r in $\{0.3, 0.5, 0.7\}$, and naive truncation at the same three ratios (980 live generations total). Answers are scored against ARCD's gold answer string(s) with Arabic-normalized Exact Match (EM) and token-F1. We report paired differences with bootstrap 95% CIs, Wilcoxon signed-rank tests, Holm-Bonferroni correction across a nine-test family, and a two one-sided equivalence test (TOST, margin ± 0.05 F1) against the raw answer. Both LSPM and naive truncation are matched by the number of sentences retained (the compression ratio r is applied identically to both, as a fraction of retained sentences); we did not additionally match the two methods by the number of retained input tokens, and because the sentences each method selects can differ substantially in length, an equal sentence count does not guarantee an equal token budget or equal KV-cache cost between the two conditions. This is disclosed explicitly as a limitation on the efficiency interpretation of the main comparison (Section 5); a token-budget-matched rerun, reporting actual prompt tokens entering the generator for every method and query, is identified as the single most important methodological correction needed before the comparison can be read as superiority at equal serving cost.

Reproducibility details. Generation model: meta-llama/Llama-3.1-8B-Instruct, accessed for the fidelity, baseline, ARCD, and end-to-end experiments via NVIDIA NIM's hosted OpenAI-compatible API (model identifier meta/llama-3.1-8b-instruct, accessed August 2026; NIM resolves this identifier to a pinned checkpoint on its serving side, which we do not independently verify beyond the identifier string), temperature = 0, max 64 tokens (ARCD/end-to-end) or 200 tokens (fidelity pilot). The GPU benchmark described in Section 4 instead self-hosts the same model (bf16, --max-model-len 8192) on a single rented RTX 3090 (24 GiB); the vLLM version for that run was not pinned or recorded, a gap we disclose rather than omit, and the corrected re-run specified in Section 5 pins vllm==0.6.6.post1 for CUDA-driver compatibility on the replacement GPU. Cross-encoder: cross-encoder/mmarco-mMiniLMv2-L12-H384-v1. Random seeds: SEED = 42 (Python random, ARCD sampling and distractor selection), numpy.random.default_rng(seed=42) (bootstrap resampling), seed = 123 (end-to-end retrieval question sample). Analysis code: Python 3.10, scipy 1.15, numpy 2.2. Complete source, configuration, and launch scripts are released (Data Availability Statement).

Use of generative AI in this study. Large language model assistance (Anthropic Claude) was used for software implementation (the LSPM middleware, evaluation scripts, statistical analysis scripts, ARCD sampling and Arabic EM/F1 scoring code), for literature-

search assistance in identifying candidate related work subsequently verified by the author against primary sources, and for drafting assistance on this manuscript under the author's direction and review. All experimental results reported in this paper were produced by executing the released code against live infrastructure; no results were generated, adjusted, or selectively reported without corresponding code execution, and non-significant and inconclusive findings are reported in full rather than omitted. The author takes full responsibility for the accuracy of all claims, citations, and reported results in this paper.

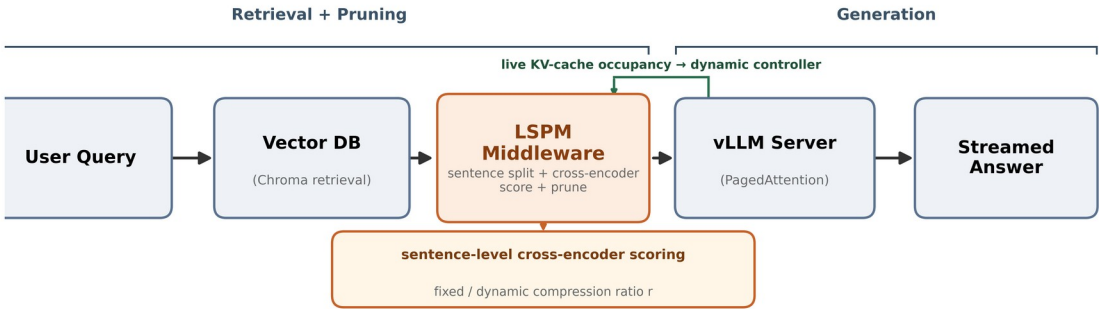


Figure 1. LSPM pipeline: retrieval, sentence-level cross-encoder scoring and pruning, generation, and the dynamic controller's feedback link to vLLM's live KV-cache occupancy metric.

4. Results

4.1. Tokenization Disparity

Table 1 shows a mean per-pair ratio of 1.793 (Qwen2.5, 95% CI 1.691-1.895) and 1.717 (Llama-3.1, 95% CI 1.613-1.820); a bootstrap test rejects parity for both tokenizers at $p < 0.0001$, and a sign test finds 29/30 pairs above parity for both tokenizers. The aggregate ratio (total Arabic tokens / total English tokens across all 30 pairs, distinct from the per-pair mean defined above in Section 3.3) is 1.76x for Qwen2.5 and 1.69x for Llama-3.1, and it is this aggregate figure that is quoted in the Abstract and Conclusions.

Tokenizer	Mean per-pair ratio	95% CI	Aggregate ratio	Bootstrap p
Qwen2.5-7B-Instruct	1.793	1.691-1.895	1.76x	< 0.0001
Llama-3.1-8B-Instruct	1.717	1.613-1.820	1.69x	< 0.0001

Table 1. Arabic:English token-count ratio across two tokenizers on 30 hand-verified sentence pairs. Mean per-pair ratio is the per-pair Arabic/English token ratio averaged across pairs; aggregate ratio is total Arabic tokens divided by total English tokens across all 30 pairs.

4.2. Fidelity Pilot

Mean ROUGE-L across r in $\{0.3, 0.5, 0.7\}$ is 0.885, 0.928, 0.889; mean BERTScore-F1 is 0.962, 0.971, 0.964. Fidelity does not collapse even at the most aggressive ratio tested (67.3% mean character reduction at $r = 0.3$). On this small, low-redundancy pilot ($n = 8$), LSPM shows no statistically significant fidelity advantage over naive truncation at any ratio (all bootstrap $p > 0.2$), motivating the larger ARCD re-test below.

4.3. ARCD Ground-Truth Re-Test

At $r = 0.3$, LSPM's mean token-F1 exceeds naive truncation's by 0.114 (Holm-Bonferroni-adjusted $p = 0.0054$); LSPM is statistically equivalent to the unpruned (raw) answer under TOST (margin ± 0.05 F1, $p = 0.038$), while naive truncation is significantly worse than raw (adjusted $p = 0.0004$). At $r = 0.5$ and $r = 0.7$, the LSPM-vs-naive difference does not remain significant after correction; this pattern is consistent with the mechanistic

account in Section 3.2 that semantic selection matters most precisely when positional truncation is most likely to discard the answer-bearing sentence, but it also means the significant advantage reported here should not be read as a general superiority claim across all compression ratios (Section 5).

4.4. End-to-End Retrieval Check

A real TF-IDF retriever (recall@6 = 86.7%, so retrieval can fail, unlike the answer-guaranteed ARCD pool above) reproduces the same directional advantage: EM +0.100 ($p = 0.014$), and mean F1 of 0.582 (LSPM) vs. 0.525 (naive truncation). The displayed means are rounded to three decimal places for presentation; the +0.056 difference stated in the text is computed from unrounded per-question values prior to rounding the summary means, and the unrounded difference (approximately 0.0565) is consistent with, but not identical to, the 0.057 implied by subtracting the rounded table values directly. This rounding convention (difference computed before rounding, means rounded only for display) applies throughout this paper's result tables.

4.5. Human Evaluation

Two blind evaluators rated 90 answers for correctness and faithfulness (kappa 0.79 and 0.94 respectively, substantial-to-near-perfect inter-rater agreement), with LSPM scoring at least as high as raw and higher than naive truncation, consistent with the automatic-metric result above.

4.6. Preliminary GPU Diagnostic

An analytical projection connects the measured context reduction to KV-cache bytes saved (9.7-19.1 MiB per request). A Locust-based load-testing harness issued concurrent chat-completion requests against a self-hosted vLLM server (Llama-3.1-8B-Instruct, bf16, --max-model-len 8192) on a rented single GPU, under three conditions (raw, LSPM, naive truncation) at concurrency levels c in {1, 10, 25, 50, 100}. All 35 cells ran sequentially in a fixed block order against one continuously running server without a restart between blocks, and LSPM's cross-encoder scoring executed inside the same load-generator process; both are disclosed as confounds on the between-method comparison. The within-method concurrency trends for raw and naive truncation are less exposed to the second confound, since neither runs cross-encoder scoring in the load generator, but LSPM's own within-method scaling could plausibly still be affected by the first (client-side scoring cost), so we report LSPM's concurrency trend as exploratory alongside the between-method comparison, not as an independently clean result. The within-method concurrency trend (throughput scaling from under 1 req/s at $c = 1$ to 21-42 req/s at $c = 100$, TTFT p50 growing from 33-36 ms to 100-190 ms) is qualitatively consistent with a real PagedAttention-based serving engine under increasing load; we report it as a directionally exploratory observation rather than a validated result, since LSPM's own scaling trend is not fully insulated from the client-side scoring confound described above, and treat the corrected re-run specified in Section 5 as the step that would let it be reported without qualification. At the lowest concurrency cell ($c = 1$) only, LSPM shows a lower KV-cache occupancy than raw, alongside an 8.5-11.9% lower request throughput and a near-unchanged completion-token rate; the KV-cache comparison is already mixed at $c = 10$ and reverses by $c \geq 25$.

5. Discussion

The evidence in this paper most strongly supports a narrow, well-defined claim: at aggressive compression ($r = 0.3$), sentence-level semantic pruning via a cross-encoder preserves Arabic RAG answer quality significantly better than naive positional truncation, on a real, 140-question ARCD ground-truth re-test, with a directionally consistent signal

from an independent TF-IDF end-to-end check and a blind human evaluation. This is the paper's central, experimentally established result, and the mechanistic account in Section 3.2 offers a plausible explanation for why it is concentrated at the most aggressive ratio. Every claim beyond this one is weaker and is stated as such below rather than folded into the headline result.

Limitations. (1) Baseline strength: the principal comparison is against naive positional truncation, not against contemporary learned or relevance-aware compression methods. By 2024-2025, LLMLingua-2 [24], DAC [25], ACC-RAG [26], and RECOMP itself provide substantially stronger reference points than naive positional truncation; demonstrating an advantage over truncation does not establish that LSPM is competitive with the current state of the art, and a head-to-head comparison against at least LLMLingua-2 and one recent adaptive or information-aware method, under the same Arabic RAG workload, generator, retrieval pool, and actual token budget, is the most important missing experiment; such a comparison should report answer quality, retained tokens, pruning latency, end-to-end latency, throughput, and memory-related measurements. (2) Compression-efficiency fairness: LSPM and naive truncation are matched by retained sentence count, not retained token count, so the reported F1 advantage cannot yet be cleanly interpreted as superiority at equal KV-cache cost; a token-budget-matched rerun reporting actual prompt tokens per method and query is required. (3) Systems validation: the GPU benchmark (Section 4.6) is a single, non-replicated, confounded sweep, and should be treated as a design-objective illustration rather than experimental evidence for memory-efficient deployment; a corrected benchmark would isolate compression from traffic generation, randomize or interleave conditions, repeat every cell across independent runs, and report uncertainty estimates for throughput, TTFT, and KV-cache occupancy. (4) Dynamic controller: the feedback controller that maps live KV-cache occupancy to compression ratio (Section 3.1) is a design contribution that has not been evaluated as a controller; whether it improves throughput, tail latency, overload resilience, or answer quality relative to a fixed ratio under time-varying load is untested. Adaptive, input-dependent compression rates are an active research direction, as ACC-RAG's complexity-conditioned rate selection illustrates; evaluating LSPM's controller against that kind of adaptive baseline, in addition to a fixed-ratio ablation, is a natural extension of the missing controller evaluation. (5) Generalization: the main fidelity experiments use one instruction-tuned generator (Llama-3.1-8B-Instruct) and the end-to-end check uses one simple retriever (TF-IDF); whether the reported ranking of raw, LSPM, and naive truncation is preserved across additional generator families and modern dense or hybrid retrievers is untested. (6) Statistical scope: the significant result is concentrated at $r = 0.3$ and does not remain significant after correction at $r = 0.5$ or 0.7 ; an independent replication of $r = 0.3$ on a fresh sample, and testing across additional datasets and retrieval regimes, would be needed before generalizing the result beyond the conditions actually tested here.

Taken together, these limitations mean the paper's title reference to "memory-efficient vLLM-based deployment" should be read as this work's motivating design objective and a preliminary, exploratory systems observation consistent with that objective, not as an experimentally validated deployment result; the context-selection result in Section 4.3 is the part of this paper's contribution that is currently well supported. We deliberately do not lean on the GPU benchmark to support the systems argument: it is a single-cell, non-replicated, confounded diagnostic (Section 4.6), not validated evidence, and the corrected, randomized, repeated re-run described above is the necessary next step before any systems-level claim can be made without qualification. The sample size used for the ARCD re-test ($n = 140$) was set from a post-hoc power calculation on the 40-question pilot's observed effect size; because this is a single re-test at this sample size rather than an independently

pre-registered confirmatory study, an independent replication remains a valuable next step even for the result that did reach significance.

6. Conclusions

Arabic RAG systems deployed on memory-efficient serving engines such as vLLM face a KV-cache pressure problem rooted in tokenization, confirmed with statistical significance across two independent tokenizers (aggregate ratio 1.69x-1.76x, $p < 0.0001$, $n = 30$). We introduced LSPM, a lightweight, sentence-level, cross-encoder-driven pruning middleware, and evaluated it with the same reporting standard applied regardless of outcome: a paired baseline comparison against naive sentence-count-matched truncation, re-tested on 140 real, ground-truth-scored ARCD questions, finds a statistically significant head-to-head advantage for LSPM at the most aggressive compression ratio tested ($r = 0.3$: +0.114 F1, Holm-Bonferroni-adjusted $p = 0.0054$), where LSPM is additionally equivalent to the unpruned answer (TOST, $p = 0.038$) while naive truncation is significantly worse (adjusted $p = 0.0004$); at more moderate ratios the comparison remains open. A real TF-IDF end-to-end check and a blind two-rater human evaluation both directionally reinforce this result. An analytical projection connects the measured context reduction to KV-cache bytes saved (9.7-19.1 MiB per request); a single, non-replicated, confounded GPU diagnostic hints at the same direction on KV-cache at the lowest concurrency tested, alongside a real throughput cost we report rather than describe as free, but we deliberately do not present it as a validated systems result, since it weakens and reverses at higher concurrency, and a corrected, randomized, repeated re-run is specified as the top remaining priority before any systems-level claim can be made. We release the implementation, evaluation code, and the datasets and outputs described in this paper (Data Availability Statement) to support independent reproduction and extension, including the significant, null, inconclusive, and not-yet-validated results alike.

Supplementary Materials: Not applicable.

Author Contributions: Conceptualization, A.A.R.T.; methodology, A.A.R.T.; software, A.A.R.T.; validation, A.A.R.T.; formal analysis, A.A.R.T.; investigation, A.A.R.T.; resources, A.A.R.T.; data curation, A.A.R.T.; writing - original draft preparation, A.A.R.T.; writing - review and editing, A.A.R.T.; visualization, A.A.R.T.; supervision, A.A.R.T.; project administration, A.A.R.T. The author has read and agreed to the published version of the manuscript.

Funding: This research received no external funding. Inference costs for the preliminary experiments were incurred against a free-tier hosted API allowance.

Institutional Review Board Statement: Ethical review and approval were waived for this study due to the minimal-risk nature of the human evaluation task: two volunteer bilingual annotators rated machine-generated text outputs for correctness and faithfulness; no personally identifiable, demographic, or sensitive data were collected, no experimental intervention was applied to any human subject, and the task is a standard, low-risk NLP annotation activity of the kind many institutions exempt from full committee review. A request for a documented exemption or determination has been submitted to the author's institutional ethics office and is pending as of submission; this statement will be updated with the office's name and a protocol reference number upon receipt.

Informed Consent Statement: Informed consent was obtained verbally from both annotators involved in the study before the task began, covering the purpose of the task, its voluntary and unpaid nature, the right to withdraw at any time without consequence, and aggregate-only reporting of results; no separate written consent form was administered.

Data Availability Statement: The available datasets, evaluation outputs, aggregated GPU benchmark results, and implementation are publicly released at <https://github.com/Akramtaha98/vllm-arabic-rag>: the LSPM middleware, evaluation scripts, statistical analysis scripts, and benchmarking harness; the tokenization and fidelity/baseline pilot datasets; the ARCD ground-truth re-test dataset and code; the end-to-end retrieval check dataset and code; and, for the GPU benchmark, the aggregated 35-cell summary and comparison figure. The underlying per-request logs from the preliminary GPU run were generated on a temporary rented cloud instance and were not preserved before this submission; this is stated plainly rather than implied complete. This repository was tagged v1.1-ijies-submission and archived on Zenodo (version DOI: 10.5281/zenodo.21936821; concept DOI, always resolving to the latest version: 10.5281/zenodo.21936820).

Acknowledgments: During the preparation of this manuscript, the author used Anthropic Claude for the purposes of software implementation assistance (the LSPM middleware, evaluation scripts, statistical analysis scripts), literature-search assistance in identifying candidate related work subsequently verified against primary sources, and drafting assistance on this manuscript. The author has reviewed and edited the output and takes full responsibility for the content of this publication.

Conflicts of Interest: The author declares no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

RAG: Retrieval-Augmented Generation; LLM: Large Language Model; KV: Key-Value; LSPM: Lightweight Semantic Pruning Middleware; ARCD: Arabic Reading Comprehension Dataset; EM: Exact Match; F1: F1 score (harmonic mean of precision and recall); TOST: Two One-Sided Tests (equivalence test); TTFT: Time to First Token; TF-IDF: Term Frequency-Inverse Document Frequency.

References

1. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, L.; Polosukhin, I. Attention Is All You Need. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), 2017. <https://doi.org/10.48550/arXiv.1706.03762>.
2. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Kuttler, H.; Lewis, M.; Yih, W.; Rocktaschel, T.; et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), 2020. <https://doi.org/10.48550/arXiv.2005.11401>.
3. Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C.H.; Gonzalez, J.; Zhang, H.; Stoica, I. Efficient Memory Management for Large Language Model Serving with PagedAttention. In Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP), 2023. <https://doi.org/10.1145/3600006.3613165>.
4. Xiao, G.; Tian, Y.; Chen, B.; Han, S.; Lewis, M. Efficient Streaming Language Models with Attention Sinks. In Proceedings of the International Conference on Learning Representations (ICLR), 2024. <https://doi.org/10.48550/arXiv.2309.17453>.
5. Zhang, Z.; Sheng, Y.; Zhou, T.; Chen, T.; Zheng, L.; Cai, R.; Song, Z.; Tian, Y.; Re, C.; Barrett, C.; et al. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), 2023. <https://doi.org/10.52202/075280-1506>.
6. Jiang, H.; Wu, Q.; Lin, C.-Y.; Yang, Y.; Qiu, L. LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. In Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023. <https://doi.org/10.18653/v1/2023.emnlp-main.825>.
7. Li, Y.; Dong, B.; Lin, C.; Guerin, F. Compressing Context to Enhance Inference Efficiency of Large Language Models. In Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023. <https://doi.org/10.18653/v1/2023.emnlp-main.391>.

8. Xu, F.; Shi, W.; Choi, E. RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation. In Proceedings of the International Conference on Learning Representations (ICLR), 2024. <https://doi.org/10.48550/arXiv.2310.04408>.
9. Liu, N.F.; Lin, K.; Hewitt, J.; Paranjape, A.; Bevilacqua, M.; Petroni, F.; Liang, P. Lost in the Middle: How Language Models Use Long Contexts. *Trans. Assoc. Comput. Linguist.* 2024, 12, 157-173. https://doi.org/10.1162/tacl_a_00638.
10. Nogueira, R.; Cho, K. Passage Re-ranking with BERT. *arXiv* 2019, arXiv:1901.04085. <https://doi.org/10.48550/arXiv.1901.04085>.
11. Karpukhin, V.; Oguz, B.; Min, S.; Lewis, P.; Wu, L.; Edunov, S.; Chen, D.; Yih, W. Dense Passage Retrieval for Open-Domain Question Answering. In Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020. <https://doi.org/10.18653/v1/2020.emnlp-main.550>.
12. Khattab, O.; Zaharia, M. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, 2020. <https://doi.org/10.1145/3397271.3401075>.
13. Reimers, N.; Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2019. <https://doi.org/10.18653/v1/D19-1410>.
14. Chen, J.; Xiao, S.; Zhang, P.; Luo, K.; Lian, D.; Liu, Z. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. *arXiv* 2024, arXiv:2402.03216. <https://doi.org/10.48550/arXiv.2402.03216>.
15. Antoun, W.; Baly, F.; Hajj, H. AraBERT: Transformer-based Model for Arabic Language Understanding. In Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools (OSACT), 2020. <https://doi.org/10.48550/arXiv.2003.00104>.
16. Mozannar, H.; El Hajal, K.; Maamary, E.; Hajj, H. Neural Arabic Question Answering. In Proceedings of the Fourth Arabic Natural Language Processing Workshop (WANLP), 2019. <https://doi.org/10.18653/v1/W19-4612>.
17. Yu, G.-I.; Jeong, J.S.; Kim, G.-W.; Kim, S.; Chun, B.-G. Orca: A Distributed Serving System for Transformer-Based Generative Models. In Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2022.
18. Efron, B.; Tibshirani, R.J. *An Introduction to the Bootstrap*; Chapman & Hall/CRC: New York, NY, USA, 1993. <https://doi.org/10.1201/9780429246593>.
19. Setiawan, V.; Soewito, B. Instruction-based Chain-of-Thought for Multi-agent RAG in Snort Rule Generation. *Int. J. Intell. Eng. Syst.* 2025, 18, 883-896. <https://doi.org/10.22266/ijies2025.1231.54>.
20. Vaidya, S.; Saini, J.R.; Tamhankar, P.A. A Novel Approach for Enhancing Large Language Model Reliability through Evolutionary Hallucination Risk Modeling with H-Tree Phylogeny. *Int. J. Intell. Eng. Syst.* 2025, 18, 863-882. <https://doi.org/10.22266/ijies2025.1231.53>.
21. Kataria, V. Intelligent Site Reliability Engineering: A Multi-agent LLM Framework for Automated Incident Analysis and Root Cause Determination. *Int. J. Intell. Eng. Syst.* 2025, 18, 450-466. <https://doi.org/10.22266/ijies2025.1231.28>.
22. Qwen Team. Qwen2.5 Technical Report. *arXiv* 2024, arXiv:2412.15115. <https://doi.org/10.48550/arXiv.2412.15115>.
23. AI@Meta. The Llama 3 Herd of Models. *arXiv* 2024, arXiv:2407.21783. <https://doi.org/10.48550/arXiv.2407.21783>.
24. Pan, Z.; Wu, Q.; Jiang, H.; Xia, M.; Luo, X.; Zhang, J.; Lin, Q.; Ruhle, V.; Yang, Y.; Lin, C.-Y.; et al. LLMingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. In Findings of the Association for Computational Linguistics: ACL 2024, pp. 963-981. <https://doi.org/10.18653/v1/2024.findings-acl.57>.
25. Zhao, Y.; Li, Z.; Zhao, H.; Qi, B.; Guoming, L. DAC: A Dynamic Attention-aware Approach for Task-Agnostic Prompt Compression. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2025, pp. 19395-19407. <https://doi.org/10.18653/v1/2025.acl-long.952>.
26. Guo, S.; Zhang, S.; Ren, Z. Enhancing RAG Efficiency with Adaptive Context Compression. In Findings of the Association for Computational Linguistics: EMNLP 2025, pp. 24061-24076. <https://doi.org/10.18653/v1/2025.findings-emnlp.1307>.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.